

프로젝트 기획 및 설계



0. 준비작업

❖github 리포지토리 생성

- 리포지토리명 : team-caltalk
- 가시성 : public

❖자신의 컴퓨터에서 git clone

- <https://github.com/사용자명/team-caltalk>

❖ Cursor 또는 VSCode로 git clone한 폴더 오픈

❖서브에이전트 준비

- <https://github.com/VoltAgent/awesome-claude-code-subagents>

1. 도메인 정의서

❖주제 선정

- "Team CalTalk" : 팀 단위 캘린더 기반 일정 관리 + 채팅 + 일정 변경 승인

❖문제 정의 :

- 팀마다 일정이 흩어져 있고 (개인 캘린더/메신저/엑셀), 충돌·누락이 잦다.
- 일정과 대화가 분리돼 맥락 회수가 어렵다.

❖도메인 정의서 작성

- 도메인 정의서란?
 - 특정 업무 · 산업 · 제품이 다루는 문제 영역(도메인) 을 공통 언어로 정리해, 범위·개념·규칙·경계를 명확히 하는 기준 문서
- 사용 목적
 - 공통 이해 정렬: 팀·이해관계자 간 용어와 관점을 통일
 - 모호성 제거: 요구사항·설계 논의의 기반 제공
 - 변경 관리: 범위 크리프(scope creep) 방지, 영향도 파악 용이

1. 도메인 정의서

■ 도메인 정의서의 핵심 항목

- 비즈니스 컨텍스트
 - 목적과 비전, 범위, 경계 설정
 - 이해관계자 매핑
- 도메인 언어
 - 용어 사전
- 도메인 모델
 - 엔티티, 값 객체, 애그리게이트(Aggregate)
 - 핵심 비즈니스 개념의 구조화
- 비즈니스 규칙
 - 불변 규칙, 정책, 검증 규칙
 - 비즈니스 제약사항
- 업무 프로세스
 - 핵심 워크플로우와 주요 사용 사례

* Aggregate?

데이터 변경의 단위로 다루어지는 연관된 도메인 객체들의 묶음 단위로써 일관성 경계를 정의하는 DDD의 핵심 패턴

1. 도메인 정의서

- 도메인 정의서의 권장 항목
 - 도메인 이벤트
 - 중요한 비즈니스 사건들
 - 이벤트 흐름과 의존성
 - 경계 컨텍스트
 - 도메인 영역 구분
 - 컨텍스트 간 통합 방식
 - 데이터 모델
 - 개념적 데이터 구조
 - 데이터 카탈로그와 품질 규칙
 - 비기능 요구사항
 - 성능, 가용성, 보안
 - 측정 가능한 품질 기준
 - 예외 처리
 - 에지 케이스
 - 장애 대응 시나리오

1. 도메인 정의서

■ 프롬프트 예시

너는 최고의 비즈니스 도메인 전문가야. "Team CalTalk"라는 이름의 팀 단위 캘린더 기반 일정관리 + 채팅 애플리케이션을 하려고 해. 다음 내용을 바탕으로 적절한 서브에이전트를 활용해 도메인 정의서 초안을 작성해줘.

문제 정의 :

- 팀마다 일정이 흩어져 있고(개인 캘린더/메신저/엑셀), 충돌·누락이 잦다.
- 일정과 대화가 분리돼 맥락 회수가 어렵다.
- 팀장이 개인의 일정을 직접 제어하기 힘들다.

Team CalTalk 앱의 주요 기능 :

- 월, 주, 일 단위의 팀 단위 일정 관리
- 팀 일정을 함께 팀원들간 공유
- 캘린더와 채팅창을 함께 보면서 일정 변경
- 팀장은 팀 일정 추가,수정할 수 있지만, 팀원은 팀일정을 수정할 수 없음
- 팀원은 팀 일정을 볼수만 있음. 팀원이 자신과 관련된 일정을 변경하고 싶다면 채팅으로 변경을 요청함.
- 채팅은 해당일자 별로 이력을 남김
- 모든 사용자는 인증(로그인)후 애플리케이션의 기능을 이용함

필수 사항 :

- 도메인 정의서에는 핵심적인 내용만 포함하도록 함
- 작성된 도메인 정의서를 docs/ 디렉토리에 1-domain-definition.md 파일로 저장함

1. 도메인 정의서

❖도메인 정의서 품질 기준

- 완전성: 모든 핵심 영역 포함
- 명확성: 모호함 없는 명확한 정의
- 추적성: 요구사항과 구현 연결
- 검증 가능성: 테스트 가능한 기준
- 유지보수성: 지속적 업데이트 가능

❖도메인 정의서 평가 또한 AI를 이용해서 진행

▪ 프롬프트 예시

전문 서브에이전트를 이용해 다음 5가지 항목을 기준으로 방금 작성한 도메인 정의서를 평가해줘

- 완전성: 모든 핵심 영역 포함
- 명확성: 모호함 없는 명확한 정의
- 추적성: 요구사항과 구현 연결
- 검증 가능성: 테스트 가능한 기준
- 유지보수성: 지속적 업데이트 가능

1. 도메인 정의서

❖도메인 정의서 평가 결과 확인

▪ 결과를 확인하고 도메인 정의서를 다시 개선함

- **business-analyst**(도메인 정의서 품질 평가)
└ Done (0 tool uses · 14.5k tokens · 36.1s)

- Team CalTalk 도메인 정의서에 대한 5가지 기준 평가가 완료되었습니다.

📊 **종합 평가: 4.2/5.0 (우수)**

주요 평가 결과:

🟢 우수한 영역:

- 완전성 (5/5): 모든 핵심 도메인 영역을 빠짐없이 포함
- 유지보수성 (5/5): 모듈화된 구조로 향후 변경사항에 유연하게 대응 가능

🟡 양호한 영역:

- 명확성 (4/5): 대부분 명확하나 "일정 충돌", "실시간" 등 일부 개념의 구체적 정의 필요
- 추적성 (4/5): 요구사항과 도메인 모델 간 매핑이 잘되어 있으나 기술 구현 연결점 보강 필요

🔴 개선 필요 영역:

- 검증 가능성 (3/5): 정량적 기준(성능, 제약조건)과 예외 상황 처리 정의 부족

핵심 개선 제안:

1. 일정 충돌 정의 구체화 - 시간 겹침과 참가자 중복 기준 명시
2. 성능 기준 추가 - 응답시간, 동시 사용자 수 등 측정 가능한 지표
3. 예외 상황 정의 - 오류 처리, 동시 접근 등 에지 케이스 대응 방안

이러한 개선사항을 반영하면 4.8/5.0 수준의 매우 우수한 도메인 정의서가 될 것으로 평가됩니다.

- 핵심 개선 제안을 모두 적용하여 도메인 정의서를 업데이트했습니다.

📄 적용된 개선사항:

1. 명확성 개선 ✅

- 일정 충돌 정의 구체화: 수학적 조건식으로 명확한 기준 제시
- 실시간 메시지 기준: 1초 이내 전달 보장으로 구체화

2. 검증 가능성 강화 ✅

- 성능 요구사항 추가: 응답시간, 동시사용자, 처리용량 등 측정 가능한 지표
- 데이터 제약 조건: 각 필드별 구체적인 길이/형식 제한 사항

3. 예외 상황 처리 정의 ✅

- 시스템 오류: 네트워크 장애, 서버 오류, 동시 접근 충돌 대응
- 비즈니스 예외: 팀장 부재, 일정 충돌, 메시지 전달 실패 처리
- 보안 예외: 무단 접근, 권한 위반, 비정상 활동 대응

📊 개선 결과:

- 이전 평가: 4.2/5.0 → 예상 개선 평가: 4.8/5.0
- 특히 검증 가능성이 3/5 → 5/5로 대폭 향상
- 명확성과 추적성도 각각 한 단계씩 개선

1. 도메인 정의서

❖ AI가 생성한 도메인 정의서를 검토하고 수정함.

- AI가 생성한 도메인 정의서의 내용이 개발자의 의도와 일치하지 않을 수 있으므로 각 항목을 검토하면서 도메인 정의서를 수정할 수 있어야 함
- 도메인 정의서를 직접 수정후에 다시 도메인 정의서를 평가하는 과정을 거침

2. PRD

❖PRD(Product Requirement Document)란?

- '무엇을? 왜?' 만들지를 명시한 제품 요구 사항 문서
- 비즈니스 문제와 목표, 성공지표, 범위·우선순위를 명확히 하여 이후 설계·개발의 기준점이 됨
- 모든 이해관계자가 제품 비전과 요구사항을 공유하는 단일 소스(SSOT:Single Source Of Truth)

❖도메인 정의서 vs PRD

- 도메인 정의서 : "무엇을"과 "왜"를 명확히 함
- PRD : 도메인 정의서 내용에 "어떻게"와 "언제"가 추가되어야 함
- 도메인 정의서는 PRD의 뼈대를 제공함. 하지만 살을 붙이는 추가적인 작업이 요구됨

❖도메인 정의서가 커버되는 PRD 요소

- 제품 개요 및 목표
- 핵심 기능 요구사항
- 사용자 스토리/유스케이스
- 기술적 제약사항 (일부)

2. PRD

❖PRD 작성을 위해 추가로 필요한 정보

- 아래와 같은 정보를 직접 작성하기 보다는 AI에게 필요한 정보를 알려주도록 요구
 - "@docs\W1-domain-definition.md 를 이용해 PRD를 작성하려고 해. 추가로 필요로 하는 정보가 있다면 알려줘"

[필수 정보]

1. 사용자 중심 정보
 - 타겟 사용자 페르소나
 - 사용자 여정 맵
 - * 온보딩 프로세스
 - * 일상적 사용 시나리오
 - * 팀 협업 시나리오
2. 기능 상세 명세
 - 사용자 스토리
 - UI/UX 요구사항
 - * 화면 레이아웃 (캘린더 + 채팅 분할 방식)
 - * 모바일/데스크톱 대응 전략
 - * 접근성 요구사항
3. 기술적 결정사항
 - 기술 스택 선택 근거
 - MVP 범위 정의

반응형 UI: 화면비율, 가로폭에 따라 GUI가 다르게 표시 (모바일/Desktop 고려)

[권장 사항]

1. 비즈니스 전략
 - 시장 분석
 - 수익 모델
2. 성과 지표
 - 비즈니스 KPI
 - 제품 성과 지표
3. 출시 전략
 - 알파/베타/정식 출시 타임라인
 - 주요 마일스톤별 목표
 - 리스크 요소 및 대응 방안

2. PRD

❖추가 정보 프롬프트 예시

전문적인 서브에이전트를 활용하여 @docsW1-domain-definition.md 내용과 다음과 같은 추가 정보를 이용하여 PRD를 2-PRD.md 에 작성해줘.

비즈니스 관련 정보

- 목표 사용자: 4-10명 수준의 팀, IT 개발팀, 스타트업, 소프트웨어 하우스에서 이용하기 위한 도구
- 비즈니스 목표: 3000개 이상의 팀이 동시에 프로젝트를 진행할 수 있도록 함
- 경쟁사 분석: 일정을 변경할 때 채팅 이력으로 근거를 남김

기술 제약사항

- 플랫폼 우선순위: 웹 우선, 모바일은 반응형 UI로만 고려함.
- 인프라 제약: 프론트엔드는 React, 백엔드는 node.js+express, 데이터베이스는 postgresql 사용
- 통합 요구사항: Google Calendar, Slack 등과 연동하는 기능은 적용하지 않음. 추후 적용 예정

프로젝트 관리

- 일정: 5일 내에 핵심 기능 완성
- 팀 구성: 1인 개발
- 예산: 1인 개발에 준하는 예산

사용자 경험

- 사용자 페르소나: 주 사용자별 상세 시나리오는 생략
- 접근성: 이 앱에서는 고려하지 않음
- 모바일 우선: 반응형 UI 적용

2. PRD

❖작성된 PRD 예시의 일부

Team CalTalk 제품 요구사항 정의서

문서 버전: 1.0 작성일: 2025-08-22 작성자: Product Management

제품 개요

제품 요약

Team CalTalk는 팀 중심의 일정 관리와 실시간 커뮤니케이션을 통합한 협업 플랫폼입니다. 기존 일정 관리 도구들과 달리, 일정 변경 요청과 의사결정 과정을 채팅 이력으로 투명하게 보존하여 근거 기반의 협업을 지원합니다.

문서 목적

이 문서는 Team CalTalk의 핵심 기능 개발을 위한 포괄적인 요구사항을 정의하며, 개발팀이 5일 내에 MVP를 구현할 수 있도록 구체적인 기능 명세와 우선순위를 제시합니다.

목표

비즈니스 목표

- 3000개 팀이 동시에 사용할 수 있는 확장 가능한 플랫폼 구축
- 팀 내 일정 조율 과정의 투명성과 추적 가능성 확보
- 기존 일정 관리 도구 대비 차별화된 커뮤니케이션 중심 접근법 제공
- IT 개발팀과 스타트업 시장에서 초기 사용자 기반 확보

사용자 목표

- 팀 일정을 한눈에 파악하고 효율적으로 관리
- 일정 변경 시 팀원 간 원활한 소통과 합의 도출
- 일정 조율 과정과 결정 근거의 투명한 기록 보존
- 실시간 커뮤니케이션을 통한 빠른 의사결정

2. PRD

❖PRD 검토/수정

- 생성된 PRD의 내용을 하나씩 검토하면서 프롬프트를 이용해 수정해나감
- 예시1 :
 - @docs₩2-PRD.md 에서 실시간 채팅은 배포되는 환경을 고려해 Websocket이 아닌 Long Polling 기법을 사용하도록 변경해줘
- 예시2 :
 - @docs₩2-PRD.md 이번 개발에서는 우선 순위 2의 커뮤니케이션 강화 기능을 제외해줘.

3. 사용자 시나리오

❖ 사용자 시나리오란?

- 특정 페르소나가 어떤 맥락으로 어떤 동기를 가지고 애플리케이션을 이용해 목표를 달성해나가는 과정을 시간의 흐름에 따라 서술한 것
- 기능 목록이 아니라 이야기 형태로 구성하여 PRD의 요구사항에 현실적인 맥락을 덧붙여줌

❖ 사용자 시나리오 작성 예시 프롬프트 예시

- 적절한 서브에이전트를 사용하여 @docsW2-PRD.md 와 @docsW1-domain-definition.md 를 참조해서 사용자 시나리오를 작성하고 3-user-scenario.md 로 저장해줘.

3. 사용자 시나리오

❖ 생성된 사용자 시나리오 예시의 일부

시나리오 1: 신규 팀 생성 및 팀원 초대

배경

김개발은 새로운 프로젝트를 위해 Team CalTalk에서 팀을 생성하고 팀원들을 초대해야 합니다.

참여자

- 김개발: 10명 개발팀의 팀장, 32세
- 이코더: 시니어 개발자, 28세 (초대받을 팀원)

시나리오 진행

1단계: 김개발의 회원가입 및 팀 생성

사용자 행동:

1. 김개발이 Team CalTalk 웹사이트에 접속
2. "회원가입" 버튼 클릭
3. 이메일(kim.dev@company.com), 비밀번호(DevTeam2024!), 이름(김개발) 입력
4. "가입하기" 버튼 클릭

4. 와이어프레임 작성

❖와이어프레임이란?

- 웹사이트, 애플리케이션의 구조와 레이아웃을 단순한 선과 도형으로 표현한 UI 설계도
- 설계 디자인이 적용되기 전에 화면에 어떤 요소들이 배치되고 어떤 방식으로 구성될지를 보여줌
- 아이디어 구체화, 개발 팀원 간의 의사소통을 돕는 역할을 함
- 프롬프트 예시
 - 와이어프레임을 작성을 위해 @docs/ 디렉토리의 어떤 문서들을 활용해야 할까?
 - 프롬프트의 결과를 이용해 와이어프레임을 작성하라고 다시 프롬프트
- 적절한 서브에이전트를 사용하여 @docs/ 디렉토리의 주요 문서를 분석한 후 와이어프레임을 4-wireframes.md 파일로 작성해줘.

❖와이어프레임 개선

- 와이어프레임 화면 하나하나를 검토하면서 업데이트를 수행함
- 프롬프트 예시
 - @docs\4-wireframes.md 에서 채팅화면과 캘린더를 한 화면에 나타내도록 해줘. 만일 화면의 크기가 두 기능을 모두 나타낼 수 없다면 채팅 기능을 Show/Hide 형태로 처리하도록 반응형 UI로 만들어줘.

4. 와이어프레임 작성

❖ 작성된 와이어프레임 문서 예시

3.3 메인 대시보드 (데스크톱)



5. 기술 아키텍처 설계

❖ 프로젝트 구조 원칙 생성 검토

- 주로 PRD 이전에 초안을 잡고, PRD 초안이 작성된 후 확정된 프로젝트 구조 원칙을 작성함
- 포함할 내용
 - 모든 스택에 공통으로 적용할 최상위 원칙
 - 의존성/레이어 원칙, 코드/명명 원칙
 - 테스트/품질 원칙, 설정/보안/운영 원칙
 - 프론트엔드, 백엔드별 디렉토리 구조
- 프롬프트 예시

하위 계층에서 상위 계층의 기능을 호출하지 않도록 규칙 필요 (단방향...)

적절한 서브에이전트를 사용해서 @docsW 디렉토리의 문서를 바탕으로 다음 지침을 참조해서 프로젝트 구조 설계 원칙을 만들고 5-project-principle.md 로 저장해줘

- 모든 스택에 공통인 최상위 원칙부터 정의
- 의존성/레이어 원칙
- 코드/네이밍 원칙
- 테스트/품질 원칙
- 설정/보안/운영 원칙
- 프론트엔드, 백엔드별 디렉토리 구조

- 생성 후 내용 검토하고 수정함

5. 기술 아키텍처 설계

❖ 기술 아키텍처 다이어그램 작성

- 시스템을 어떤 구성요소로 나누고, 구성요소간 연결, 상호작용 방식은 어떠한지를 한눈에 보여주는 시각적 문서
- 여러 이해관계자가 같은 그림을 보면서 의사소통할 수 있도록 하는 공용언어 역할 수행
- 이것은 살아 있는 문서이므로 설계 과정에서 지속적으로 업데이트될 수 있음
- 프롬프트 예시
 - 적절한 서브에이전트를 사용해서 @docs\ 디렉토리의 문서를 바탕으로 기술 아키텍처 다이어그램을 mermaid 형식으로 작성한 후 6-arch-diagram.md 파일로 저장해줘. 최대한 단순한 구조로 표현해야 해.

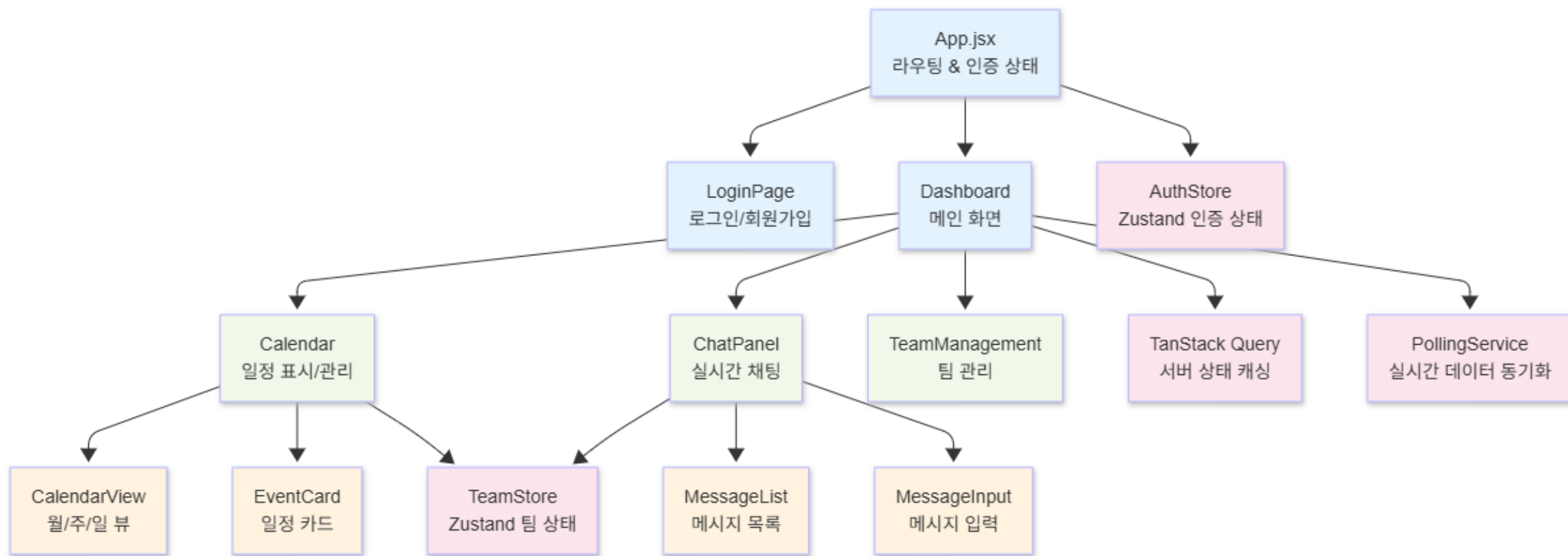
❖ PRD에 기술된 기술 스택 재검토

- 다른 문서들의 내용과 PRD의 기술 스택의 내용을 재검토하여 일관되게 작성되어 있는지 여부를 확인함.
- 프롬프트 예시
 - 적절한 서브에이전트를 사용해서 @docs\ 디렉토리에 작성된 파일을 분석하여 PRD에 작성된 기술 스택이 일관되게 작성되었는지를 검토해줘

5. 기술 아키텍처 설계

❖생성된 기술 아키텍처 다이어그램 예시

2. 프론트엔드 아키텍처 (컴포넌트 구조)



설명: React 18 컴포넌트 기반 구조로 인증, 캘린더, 채팅 기능을 독립적인 모듈로 분리. Zustand를 통한 클라이언트 상태 관리, TanStack Query를 통한 서버 상태 캐싱, Long Polling 서비스로 실시간 동기화.

5. 기술 아키텍처 설계

❖기술 스택 재검토

- @docs₩2-PRD.md 문서를 분석하여 프론트엔드, 백엔드, 데이터베이스로 구성된 앱을 개발할 때 사용할 적절한 기술 스택을 분석하고 선택해줘.
- 선택된 기술 스택을 @docs₩ 디렉토리에 있는 문서 파일에 반영해줘.
- 이 프로젝트에서 React 18을 사용하는 것과 Next.js를 적용할 때를 비교하여 장단점을 분석해줘.

🎯 성공 지표 예측

React 18 + Vite 선택 시

- ✅ 5일 MVP 달성: 95% 확률
- ✅ 성능 목표 달성: 90% 확률
- ✅ 유지보수성: 높음
- ✅ 팀 확장성: 우수

Next.js 선택 시

- ⚠️ 5일 MVP 달성: 60% 확률
- ❌ 불필요한 복잡성: 높음
- ⚠️ 러닝 커브: 가파름
- ❌ 오버엔지니어링 위험: 높음

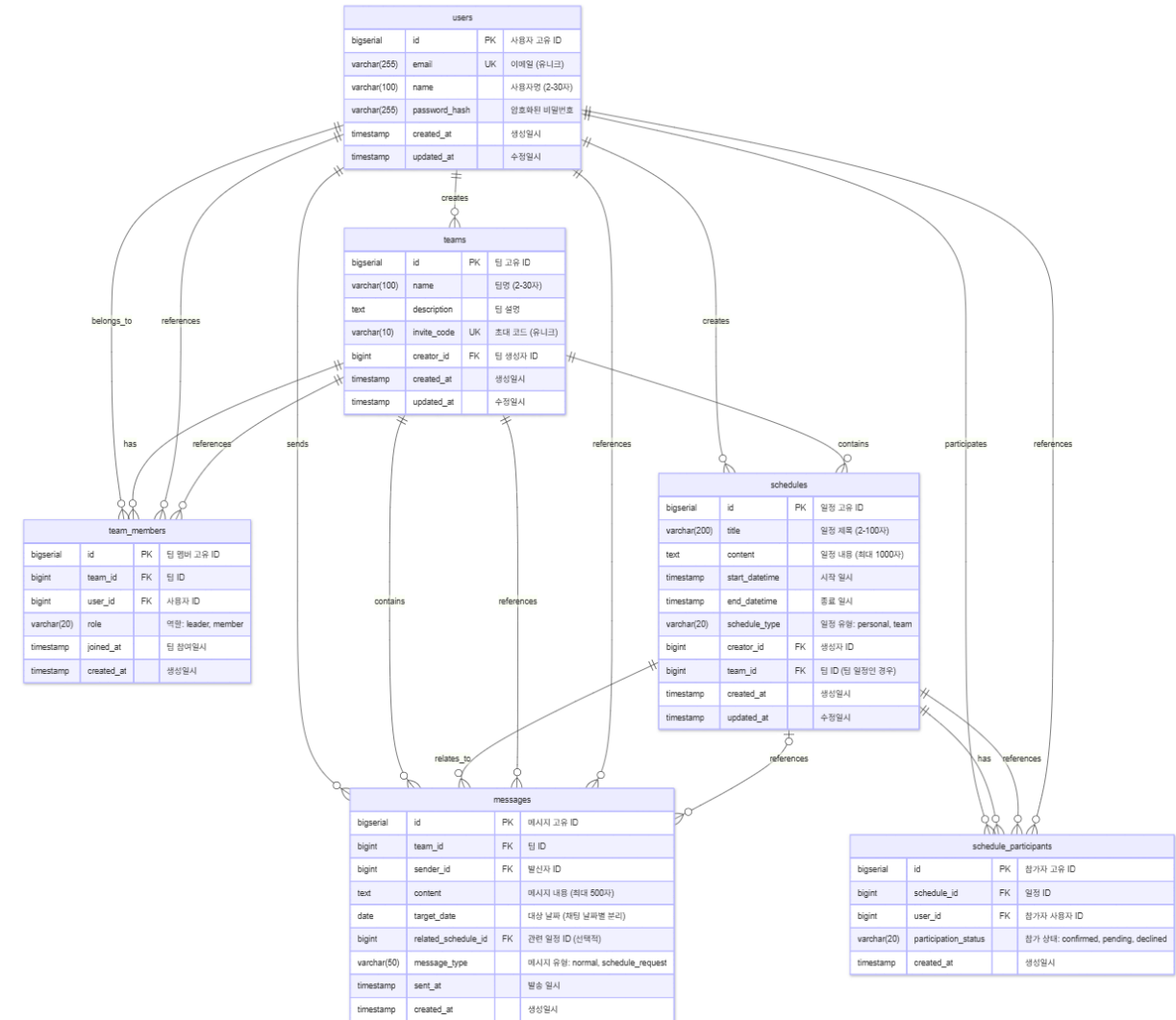
🎯 결론: React 18 + Vite가 Team CalTalk의 정답

6. ERD

❖데이터베이스 스키마 설계

- ERD 생성
- 프롬프트 예시
 - 적절한 서브에이전트를 사용해서 @docs\2-PRD.md, @docs\1-domain-definition.md 정보를 바탕으로 ERD를 mermaid 형식으로 작성하고 7-erd.md 로 저장해줘.
- 만일 아키텍처 다이어그램에 ERD가 작성되었다면 이 단계를 건너뛰

ERD (Entity Relationship Diagram)

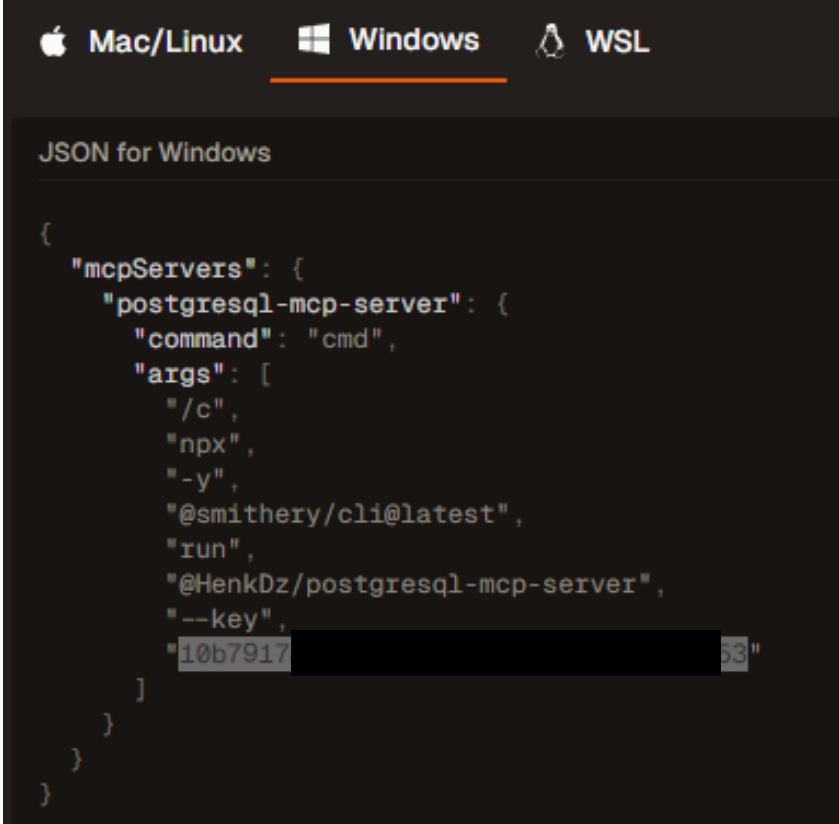


6. ERD

❖ ERD를 이용하여 데이터베이스를 생성하기 위한 설정

❖ DDL 생성 프롬프트

- @docs\7-erd.md 를 활용해 데이터베이스 생성을 위한 DDL 문을 작성해서 docs/schema.sql 파일에 저장해줘



```
Mac/Linux Windows WSL
JSON for Windows
{
  "mcpServers": {
    "postgresql-mcp-server": {
      "command": "cmd",
      "args": [
        "/c",
        "npx",
        "-y",
        "@smithery/cli@latest",
        "run",
        "@HenkDz/postgresql-mcp-server",
        "--key",
        "10b7917...53"
      ]
    }
  }
}
```


7. 작업 분할 및 실행 계획 수립

❖ 실행 계획 수립

- PRD를 기반으로 해야할 작업을 독립적인 Task 단위로 분할하고 구체적인 실행 계획을 수립함
- 반드시 포함시킬 것
 - Task별 완료조건과 의존성
 - 앞의 내용들을 체크박스 형태로 문서화하여 각 단계를 진행할 때 완료된 상태로 문서를 업데이트해 나갈 것
- 프롬프트 예시

너는 전문적인 WBS 및 실행 계획 수립 전문가이다. @docsW 디렉토리의 문서들을 검토하여 구체적인 작업 실행 계획을 수립한다. 실행 계획은 다음 사항을 준수하도록 한다.

- 데이터베이스, 백엔드, 프론트엔드 단위로 해야 할 작업들을 관리가능하고 독립적인 Task로 분해한다.
- 각 Task별 수행 작업, 완료 조건, 선행 Task가 구체적으로 명시되어야 한다.
- 완료 조건은 반드시 체크박스 형식으로 나타나야 한다.
- 수립한 실행 계획은 docs/8-pan.md 로 저장한다.

7. 작업 분할 및 실행 계획 수립

❖ 실행 결과

🎨 3단계: 일정 관리 API 및 비즈니스 로직 구현 (Day 2-3)

3.1 API 라우팅 및 미들웨어

- [] **Express** 라우터 구조 구현
 - 완료 조건: 인증, 팀, 일정, 메시지 라우터 모든 엔드포인트 정의

```
POST /api/auth/register, /api/auth/login
POST /api/teams, /api/teams/join
GET/POST/PUT/DELETE /api/teams/:teamId/schedules
GET/POST /api/teams/:teamId/messages
GET /api/teams/:teamId/messages/poll (Long Polling)
```

- 의존성: 인증 미들웨어 완료
 - 예상 시간: 2시간
- [] **입력 검증 미들웨어** 구현
 - 완료 조건: Joi 스키마 기반 검증, XSS 방지
 - 의존성: 라우터 구조 완료
 - 예상 시간: 1.5시간

8. Swagger

❖ Swagger란?

- RESTful API를 설계, 빌드, 문서화하기 위한 오픈소스 프레임워크
- API 개발자들이 API를 더 쉽게 만들고 관리할 수 있도록 도와줌

❖ 주요 기능

- **Swagger Spec** : API에 대한 표준화된 명세 제공
- Swagger UI : API Spec을 시각적으로 표현하고 테스트할 수 있는 API Console을 보여줌
- Swagger Codegen : API Spec을 이용해 클라이언트 SDK, Stub 코드를 생성해주는 도구

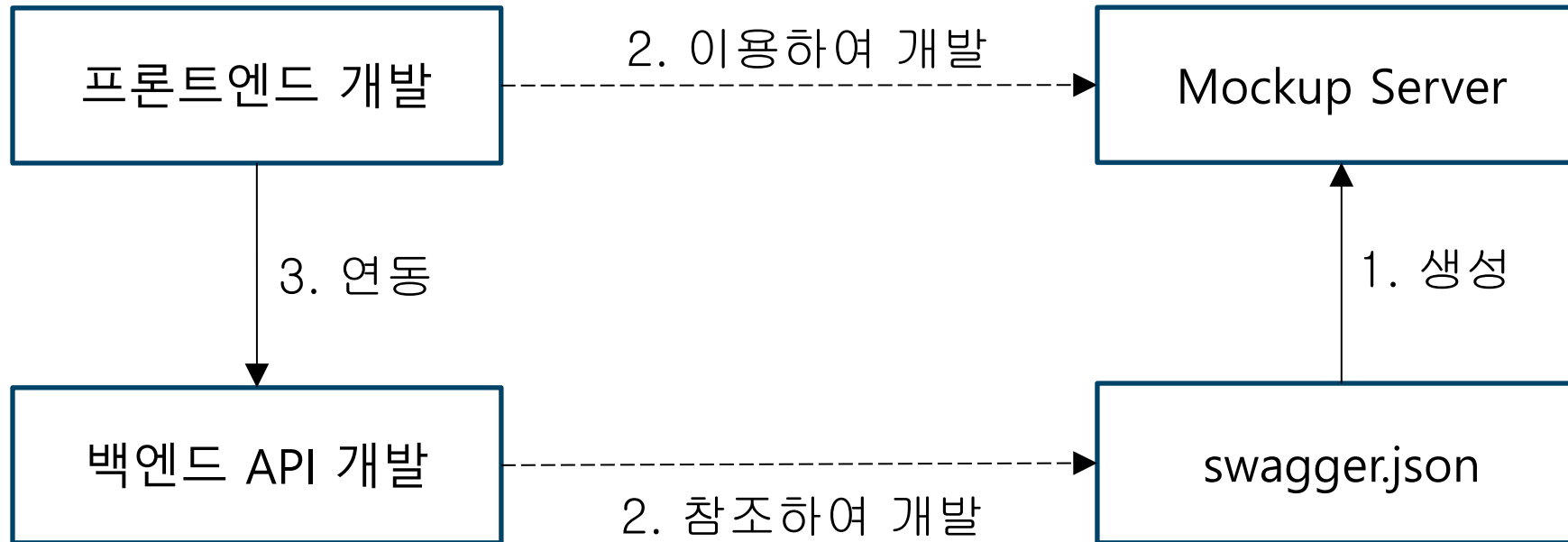
❖ 장점

- 일관성, 생산성 향상
 - 한번의 Spec 정의로 문서, 테스트, 코드 생성을 자동화 할 수 있음
- 단일 소스
 - API 변경 사항을 한 지점에서 관리
- 협업 효율 증대
 - 클라이언트와 서버간의 Contract 역할 담당

8. Swagger

❖ Swagger를 이용한 협업

- Swagger Spec을 이용해 Mockup 서버를 빠르게 생성하고 프론트엔드와 백엔드가 동시 개발



❖ 프롬프트 예시

- 적절한 서브에이전트를 사용해 docs 디렉토리에 포함된 문서를 분석하여 swagger spec을 작성하고 docs\swagger.json 파일에 저장해줘.

8. Swagger

❖ 생성된 Swagger Spec은 반드시 다시 한번 분석 검토할 것

- 적절한 서브에이전트를 이용해 docs\swagger.json 의 내용을 docs 디렉토리의 문서와 비교분석하여 일관성을 검토하고 수정해줘.

● Swagger API 명세서 일관성 검토 및 수정 완료

적용된 핵심 수정사항

1. 팀 관리 API 확장

- /api/teams/{teamId}/members/{userId} DELETE: 팀원 제거 API 추가
- /api/teams/{teamId}/invite-code/regenerate POST: 초대 코드 재생성 API 추가

2. 일정 관리 기능 보강

- category 필드 추가: meeting, deadline, personal, other
- priority 필드 추가: high, medium, low
- recurrence 객체 추가: 반복 일정 지원 (daily, weekly, monthly)

3. 실시간 채팅 기능 강화

- /api/chat/teams/{teamId}/messages/{messageId}/read POST: 메시지 읽음 처리 API
- /api/users/online-status PUT: 사용자 온라인 상태 관리 API
- 메시지 스키마에 isRead, readCount 필드 추가
- 메시지 타입 확장: system, schedule_notification 추가

4. Long Polling 이벤트 타입 확장

- schedule_conflict_detected: 일정 충돌 감지 이벤트
- message_read: 메시지 읽음 이벤트
- user_status_changed: 사용자 상태 변경 이벤트

문서 간 일관성 개선

-  PRD 요구사항 반영: 팀원 관리, 실시간 상태 표시, 일정 카테고리화
-  와이어프레임 대응: 읽음 상태, 온라인 표시, 팀 설정 기능
-  ERD 구조 일치: 데이터베이스 스키마와 API 모델 통일
-  사용자 시나리오 지원: 모든 워크플로우의 API 엔드포인트 제공

수정된 Swagger 명세서는 이제 프로젝트 문서들과 완전히 일관된 구조로 Team CalTalk의 모든 핵심 기능을 지원합니다.

8. Swagger

❖ Swagger Mockup Server

- Swagger Spec만을 이용해 실제 구현없이 API를 흉내내는 서버
- 백엔드 API가 개발 완료되기 전에도 프론트엔드 개발을 할 수 있도록 함
- 외부 고객에게 빠르게 백엔드 API를 시연할 수 있음

❖사용가능한 Mockup Server

- Stoplight Prism CLI : 가장 대표적인 도구, 샘플 데이터 자동 생성
 - <https://github.com/stoplight/prism>
- Scalar Mock Server
 - OpenAPI에서 엔드포인트를 자동 생성하고 스키마 기반 더미 데이터 생성.
 - 클라이언트 기능도 제공함
 - <https://github.com/scalar/scalar>
- Mockoon : Desktop & CLI 를 이용해 mockup server 구동
 - <https://mockoon.com>
- openapi-mock-express-middleware
 - express 프레임워크의 미들웨어를 이용해 Mockup Server를 구성할 수 있도록 해주는 도구

8. Swagger

❖ openapi-mock-express-middleware를 이용한 Mockup Server 구성

■ 프롬프트 1

mockup 디렉토리를 생성한 후 이 디렉토리로 이동한 후 다음 명령어를 실행해줘

```
npm install express openapi-mock-express-middleware swagger-ui-express nodemon
```

■ 프롬프트 2 : 서버의 Port는 swagger.json의 주소를 확인후 반영함

다음 코드로 mockup/server.js 파일을 생성해줘

```
const express = require("express");
const { createMockMiddleware } = require("openapi-mock-express-middleware");
const swaggerUi = require("swagger-ui-express");
const swaggerDoc = require("../swagger/swagger.json"); // 또는 .yaml 파싱

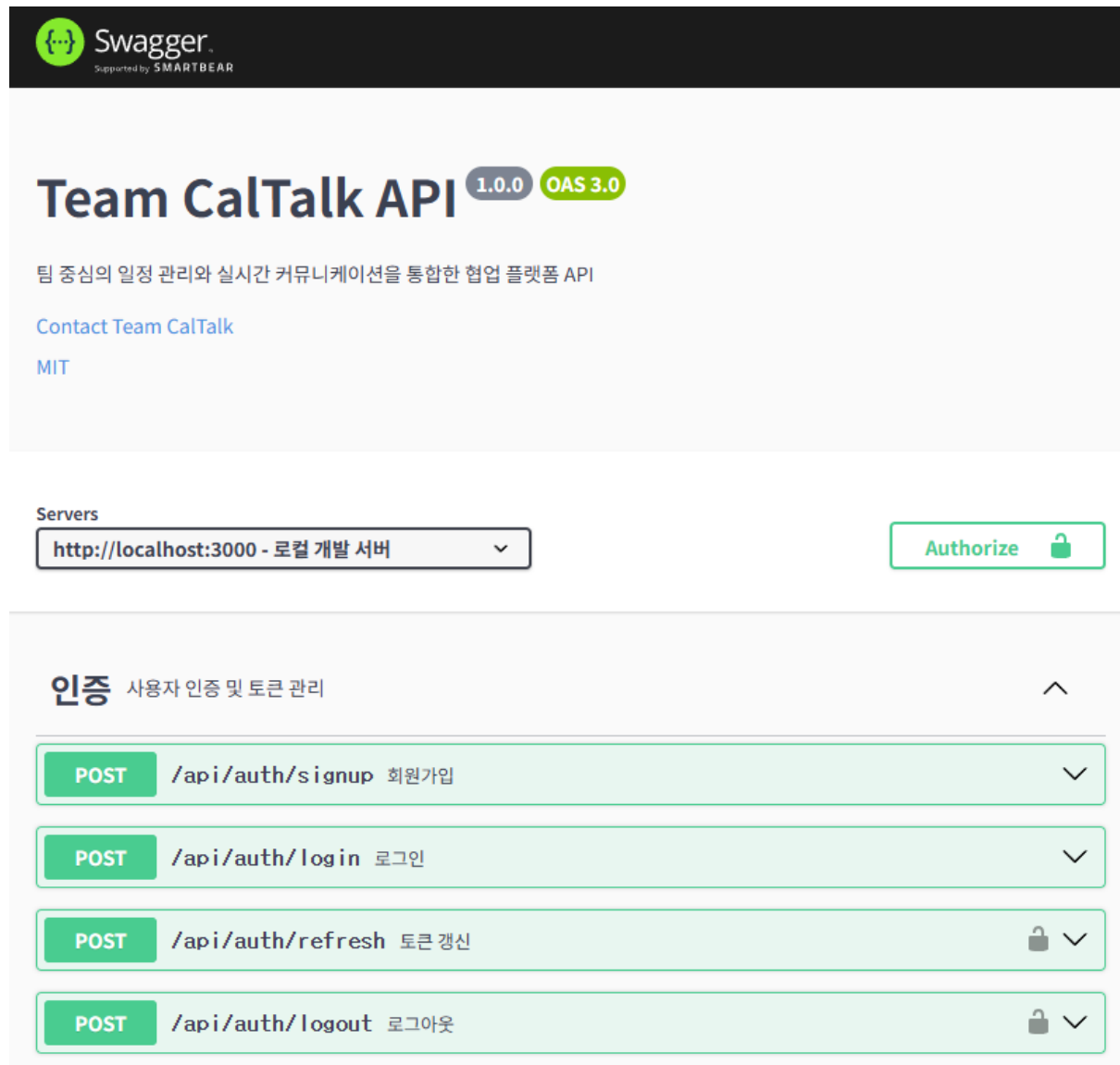
const app = express();
app.use("/api", createMockMiddleware({ spec: "../swagger/swagger.json" })); // mock 서버
app.use("/docs", swaggerUi.serve, swaggerUi.setup(swaggerDoc)); // Swagger UI

app.listen(3000);
```

8. Swagger

❖ mockup server 실행

- 다음 명령어로 서버 구동
 - `npx nodemon server.js`
- Swagger UI 확인
 - `http://localhost:3000/docs`



9. github 리포지토리 업데이트

- ❖작업이 완료되었다면 github에 업데이트를 수행한다.
 - 현재 코드베이스를 분석하고 불필요한 파일이 깃헙 리포지토리에 업데이트되지 않도록 .gitignore 파일을 생성해줘.
 - 현재까지의 상태를 git commit 하고 리모트 리포지토리에 업데이트해줘.

10. 정리

❖ 준비작업

- GitHub 'team-caltalk' 리포지토리 생성, 클론
- Cursor/VSCode 오픈, 서버에이전트 레퍼런스 준비

❖ 도메인 정의서

- 공통언어, 모호성 제거, 변경관리 목적, 컨텍스트/모델/규칙 등 핵심 항목과 이벤트,경계, 비기능 요구 포함
- AI 초안→평가→개선 루프

❖ PRD

- "무엇/왜"에서 "어떻게/언제"로 확장
- 페르소나·여정·기능 명세·기술 결정 등 추가정보 수집→초안 작성→프롬프트로 수정/검토

❖ 사용자 시나리오

- 도메인/PRD를 참조해 이야기 형태로 작성해 요구사항에 현실 맥락 부여, 와이어프레임 작성/개선

❖ 아키텍처·ERD

- 공통 원칙/디렉터리 구조 수립, Mermaid 다이어그램과 스택 일관성 검토, ERD 작성→DDL 생성

❖ 실행계획·UX·API

- Task 분해, 완료조건/의존성 체크리스트화
- Swagger Spec 작성·검토, Mock 서버(prism/Scalar/Mockoon/express 미들웨어)로 병행 개발